Karigar

Multi-Vendor Artisan Marketplace

Project Documentation

Submitted by
Laiba Idrees

DevWeekend Fellowship 2026

Overview

Karigar is a multi-vendor marketplace. Independent artisans (potters, weavers, woodworkers, leatherworkers) each run their own shop. Customers can buy from more than one shop in a single order.

The backend is a FastAPI + PostgreSQL REST API. The frontend is a React + Tailwind app.

Goals of the Project

- Build a backend with real production judgment. Not just working endpoints correct transactions, clear authorization rules, and data that stays consistent.
- Use this project as the base for future AI features (RAG, LLM APIs, vector search), instead of starting a separate project for that later.
- Practice thinking through a design before writing code, so I can explain and defend every decision, not just show that it runs.

Key Features

Multi-Role User System

- **Buyer (customer)** — browses products, adds items to a cart, checks out.
- **Seller (artisan)** — registers with a shop name. Lists and manages their own products. Sees their own orders. A new artisan account starts unapproved and cannot list products yet.
- **Admin** — approves pending artisan shops. Admin accounts cannot be created through public sign-up. They are created directly through a script.

Product Management & Shopping

- **Product listings** — artisans create, edit, and remove their own products. Removing a product is a soft delete (is_active = false), not a real delete. This way, past orders that reference the product still work.
- **Cart & checkout** — the cart lives on the frontend. But the backend never trusts the client's price or stock numbers. At checkout, the backend reads the real product rows again, locks them, and recalculates the total itself before creating the order.

Seller Dashboard Features

- **Order management** — an artisan sees only the orders placed against their own shop.
- **Product listing management** — add, edit, soft-delete, and restore their own products from a dashboard.

Functional Requirements

Authentication

- FR1 — A user can register as a customer or an artisan.
- FR2 — The system rejects self-registration with the admin role. Admin accounts are created directly, through a script.
- FR3 — A registered user can log in and receive a JWT access token.
- FR4 — An authenticated user can fetch their own profile.

Artisan Shops

- FR5 — When a user registers as an artisan, a shop profile is created for them automatically, in the same transaction as the account.
- FR6 — A new artisan shop starts unapproved and cannot list products until approved.
- FR7 — An admin can view the list of pending (unapproved) artisan shops.
- FR8 — An admin can approve a pending artisan shop.
- FR9 — Anyone can view the list of approved artisan shops and an individual shop's public profile.
- FR10 — An approved artisan can view and update their own shop profile.

Products

- FR11 — An approved artisan can create a product listing under their own shop.
- FR12 — An approved artisan can edit their own product listings.
- FR13 — An approved artisan can soft-delete (remove) and restore their own product listings.
- FR14 — The system blocks an artisan from editing or deleting a product they don't own.
- FR15 — Anyone can browse active product listings without logging in.
- FR16 — Anyone can view a single product's details directly, even if that product has since been removed from sale.

Cart & Orders

- FR17 — An authenticated customer can check out a cart containing products from one or more artisan shops.
- FR18 — The system splits a checkout into one order per artisan represented in the cart.
- FR19 — The order total is calculated on the server from current product prices, never from a value the client sends.

- FR20 — Product stock rows are locked during checkout to stop two customers from overselling the same stock.
- FR21 — Each order item stores the product's name and price as they were at the time of purchase.
- FR22 — A customer can view their own past checkouts and orders.
- FR23 — An artisan can view the orders placed against their own shop.

Non-Functional Requirements

Security

- Passwords are hashed with bcrypt before storage — never stored in plain text.
- All protected routes require a valid JWT; role and ownership are checked on the server, never trusted from the client.
- An artisan's approval status is re-checked from the database on every request, not cached in the token.

Data Integrity

- Every multi-step operation (like registration + shop creation, or checkout) commits once, as a single unit — if one step fails, nothing is saved.
- Checkout uses row-level locking to close the race condition between two customers buying the same stock at once.
- Currency fields use Numeric(10,2), not float, to avoid rounding errors in money math.
- Products are soft-deleted, so historical orders always resolve correctly even after a product is removed.

Maintainability

- The backend is a modular monolith: four independent modules, each following the same router → service → repository shape, so a new engineer can predict where any given piece of logic lives.

Usability

- API errors return specific, meaningful status codes (403, 404, 409) instead of one generic failure.
- The frontend surfaces the real reason for a failure (e.g., "insufficient stock") instead of a blank error.

Performance

- Listing endpoints are paginated instead of returning full tables.
- Foreign key columns used in lookups are indexed.

- Locking during checkout is scoped only to the rows being purchased, so normal browsing is never blocked.

Scalability

- Because module boundaries are real (not just folders), any module could be pulled out into its own service later without a rewrite of the others.

Deployability

- The database runs in Docker locally; the backend is deployed independently on Render, so local development and production don't share state.

System Architecture Overview

Karigar is built as a **modular monolith**. It is one backend, but it is split into four modules: auth, artisans, products, orders. Each module is self-contained.

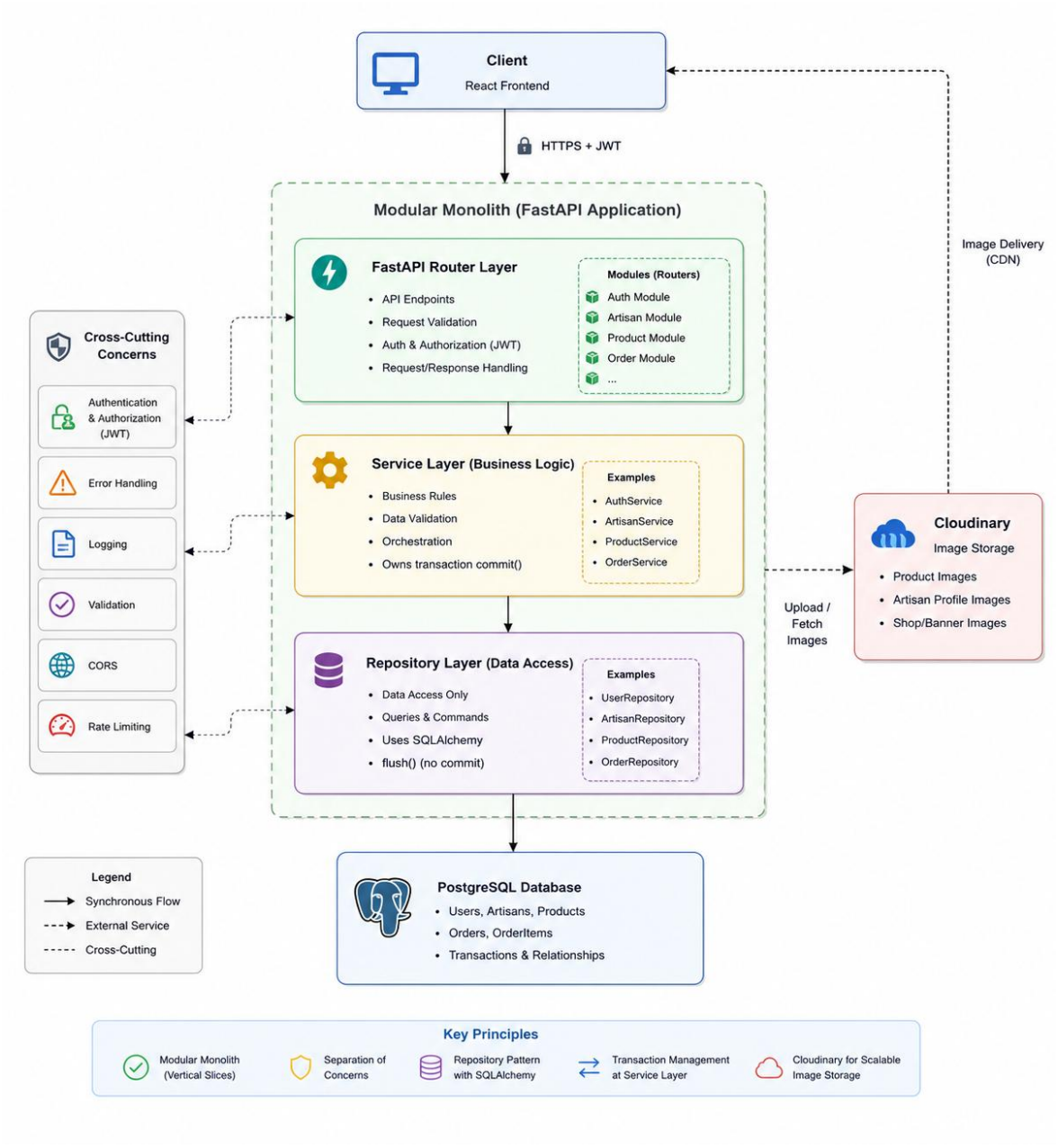
Every module follows the same layers:

Router → Service → Repository

- **Router** — handles the API request and response. Decides which auth checks apply.
- **Service** — holds the business rules. Owns the database transaction.
- **Repository** — only reads and writes rows. It does not decide who is allowed to do what.

One rule applies across the whole backend: **repositories only call flush(). Services call commit().** Flush sends the SQL but keeps the transaction open. Only the service commits, at the very end. This is what makes multi-step actions safe — for example, creating a customer account and their shop profile together. If one step fails, nothing is saved.

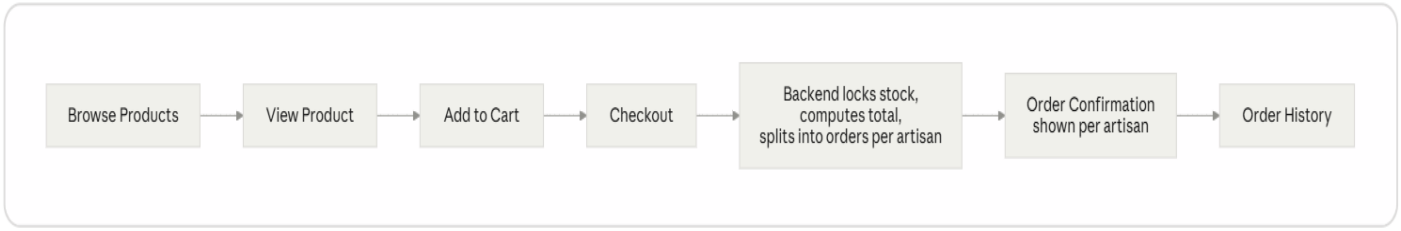
System Architecture Diagram



Application Flow Diagram

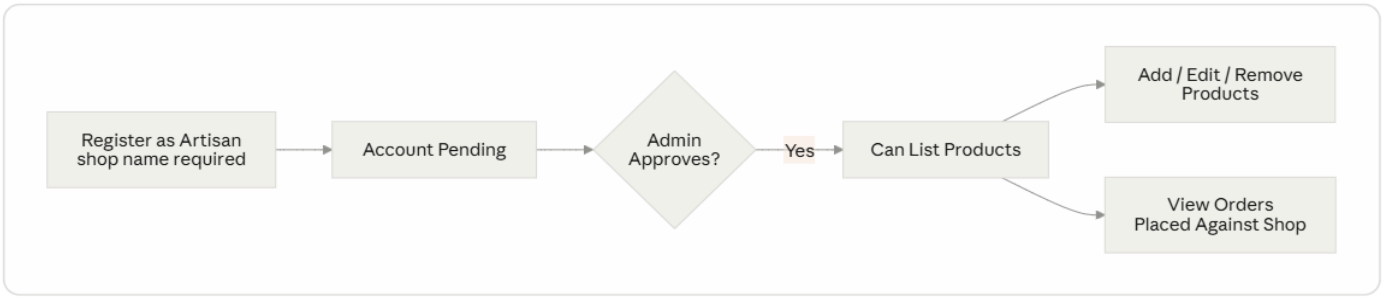
Customer Flow

A customer browses the product catalog, opens a product, and adds it to their cart. At checkout, items from different shops are split into separate orders automatically, under one shared checkout record. The confirmation screen shows this split. The customer can see they bought from two shops in one action.



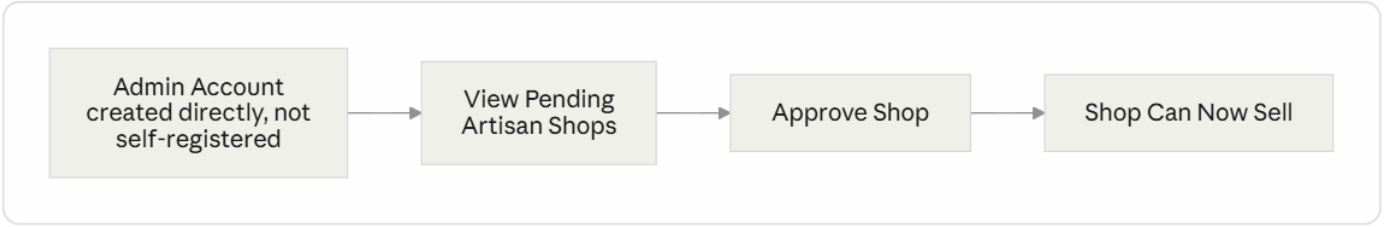
Artisan Flow

An artisan registers with a shop name. Their account exists right away, but they cannot list products until an admin approves the shop. Once approved, they get a dashboard. From there they add, edit, soft-delete, and restore their own products. They also see the orders placed against their shop — never another shop's orders.



Admin Flow

Admin accounts are not created through public sign-up. They are created directly, through a script. Letting anyone self-register as admin would be a security hole. An admin's job here is to review pending artisan shops and approve them. Approval is what lets a shop start selling.



API Architecture

Karigar exposes a REST API built with FastAPI. Endpoints are grouped by module: /auth, /artisans, /products, /orders. All data goes in and out as JSON, checked by Pydantic schemas. Protected endpoints need a JWT sent as a Bearer token. The token carries the user's identity and role. Route-level checks (require_role, require_approved_artisan) confirm who is allowed to call an endpoint before any business logic runs.

API Design

Module	Method	Endpoint	Description	Access
Authentication	POST	/auth/register	Register a new customer, artisan, or admin account.	Public
	POST	/auth/login	Authenticate a user and return an access token.	Public
	GET	/auth/me	Retrieve the currently authenticated user's profile.	Authenticated User
Artisans	GET	/artisans	List all approved artisan shops.	Public
	GET	/artisans/{artisan_id}	Retrieve the public profile of an approved artisan.	Public
	GET	/artisans/{artisan_id}/products	List all active products for a specific artisan shop.	Public
	GET	/artisans/me	Retrieve the logged-in artisan's own profile.	Approved Artisan
	PATCH	/artisans/me	Update the logged-in artisan's shop name.	Approved Artisan
	GET	/artisans/pending	List artisan shops awaiting approval.	Admin
	PATCH	/artisans/{artisan_id}/approve	Approve a pending artisan shop.	Admin
Products	GET	/products	List all active products.	Public
	GET	/products/{product_id}	Retrieve details of a specific product.	Public

Module	Method	Endpoint	Description	Access
	GET	/products/me/listings	List all products owned by the logged-in artisan.	Approved Artisan
	POST	/products	Create a new product listing.	Approved Artisan
	PATCH	/products/{product_id}	Update an owned product.	Product Owner
	DELETE	/products/{product_id}	Soft-delete an owned product.	Product Owner
	PATCH	/products/{product_id}/restore	Restore a previously soft-deleted product.	Product Owner
Orders	POST	/orders/checkout	Create a checkout for a customer. The system automatically groups cart items by artisan and creates separate orders for each artisan.	Authenticated Customer

Tech Stack

Backend

- **FastAPI** — web framework
- **PostgreSQL** — database
- **SQLAlchemy** — ORM
- **Alembic** — database migrations
- **Pydantic** — request and response validation
- **python-jose** — creates and checks JWTs
- **passlib (bcrypt)** — hashes passwords

Frontend

- **React** — UI
- **Vite** — build tool and dev server
- **Tailwind CSS** — styling
- **react-router-dom** — routing

- **axios** — API calls, through one shared client that attaches the JWT and handles 401 errors in one place

Development & Deployment Tools

- **Docker** — runs PostgreSQL locally
- **Render** — hosts the live backend
- **Vercel** — hosts the live frontend

Challenges & Solutions

Overselling risk in checkout. Two customers could both read the last unit of a product's stock at the same time. Both could pass validation. Both could buy it. This is a race condition. The fix: lock every product row involved in a checkout *before* checking stock, not after. Locking after checking would leave the same gap open.

CORS failures in production that looked like a config problem, but weren't. The environment variable for allowed origins looked correct in the dashboard. Requests still failed. The real cause: the latest code had not actually redeployed. The running process did not match what was in the dashboard or in git. Fix: a temporary debug endpoint that showed the config the live process had actually loaded. That exposed the mismatch directly, instead of guessing.

Alembic generating an empty migration. alembic revision --autogenerate compares the model file as saved on disk. If the model was edited but not saved before running the command, Alembic sees no change and makes an empty migration. Fix: always save the file first, and always read the generated migration before applying it.

Database Design

Tables: users, artisans, products, checkouts, orders, order_items

Key relationships:

- users to artisans — one to one (only artisan-role users have a row here)
- artisans to products — one to many
- checkouts to orders — one to many (one checkout can split into several orders, one per shop)
- orders to order_items — one to many

This is well structured for Markdown, but for a Word document it is usually better to use numbered sections and tables with consistent headings. Here's a polished version you can paste directly into Word.

Data Schema

The **Karigar** marketplace uses a relational **PostgreSQL** database. The schema is designed to support a multi-vendor marketplace where customers purchase products from multiple artisan shops through a single checkout, while the system automatically creates separate orders for each artisan.

Users

Field	Data Type	Description
id	UUID (Primary Key)	Unique identifier for each user
email	String (Unique)	Email address used for authentication
password_hash	String	Securely hashed password
name	String (Nullable)	User's display name
role	Enum	User role (`customer`, `artisan`, `admin`)
created_at	DateTime	Account creation timestamp
updated_at	DateTime	Last account update timestamp

Artisans

Field	Data Type	Description
id	UUID (Primary Key)	Unique artisan shop identifier
user_id	UUID (Foreign Key → Users.id)	Owner of the artisan shop
shop_name	String	Name of the artisan shop
description	Text (Nullable)	Shop description
location	String (Nullable)	Artisan shop location
is_approved	Boolean	Indicates whether the shop has been approved by an administrator

Products

Field	Data Type	Description
id	UUID (Primary Key)	Unique product identifier
artisan_id	UUID (Foreign Key → Artisans.id)	Artisan shop that owns the product
name	String	Product name
description	Text (Nullable)	Product description
price	Numeric (10,2)	Product price
stock_quantity	Integer	Available inventory
image_url	String	URL of the product image
is_active	Boolean	Indicates whether the product is publicly listed
created_at	DateTime	Product creation timestamp

Field	Data Type	Description
updated_at	DateTime	Last product update timestamp

Checkouts

Field	Data Type	Description
id	UUID (Primary Key)	Unique checkout identifier
customer_id	UUID (Foreign Key → Users.id)	Customer who completed the checkout
total_amount	Numeric (10,2)	Total value of the checkout
payment_status	Enum	Payment status (`pending`, `paid`, `failed`)
payment_reference	String (Nullable)	Payment transaction/reference identifier
created_at	DateTime	Checkout creation timestamp

Orders

Field	Data Type	Description
id	UUID (Primary Key)	Unique order identifier
checkout_id	UUID (Foreign Key → Checkouts.id)	Parent checkout
artisan_id	UUID (Foreign Key → Artisans.id)	Artisan responsible for fulfilling the order
status	Enum	Order status (`pending`, `shipped`, `delivered`, `cancelled`)
created_at	DateTime	Order creation timestamp

Order Items

Field	Data Type	Description
id	UUID (Primary Key)	Unique order item identifier
order_id	UUID (Foreign Key → Orders.id)	Associated order
product_id	UUID (Foreign Key → Products.id)	Purchased product
product_name	String	Snapshot of the product name at the time of purchase
unit_price	Numeric (10,2)	Snapshot of the product price at the time of purchase
quantity	Integer	Number of units purchased

Key design choices:

- Money fields use Numeric(10,2), never float. Floats cannot represent decimal currency exactly. This is a real correctness bug, not just a style choice.
- Product.artisan_id points to artisans.id, not users.id. The artisan (shop) is what owns approval status and product ownership, not the user account itself.
- Products are soft-deleted with an is_active flag instead of being deleted for real. Old orders can still show what was bought, even after a product is removed from sale.
- order_items stores its own copy of product_name and unit_price at the time of purchase. It does not just link to the live product. If a product's name or price changes later, old receipts still show what the customer actually saw when they bought it.

Best Practices

Authentication & Security

- JWT-based authentication. The user's role is encoded in the token.
- Passwords are hashed with bcrypt before being stored. The plain password is never saved anywhere.
- Route-level checks confirm both *who you are* (authentication) and *what you are allowed to do* (role and ownership checks), before any business logic runs.

Component Architecture

- **Backend:** each module is a self-contained slice (router, service, repository). A change in products does not spill into orders unless there is a real reason for it.
- **Frontend:** shared, reusable components (ProductCard, Navbar, ProtectedRoute), instead of repeating the same markup on every page. Page-level components handle their own data-fetching and state.

Error Handling & User Experience

- API errors return specific status codes: 403 for "not yours," 404 for "doesn't exist," 409 for conflicts like a duplicate email. Not one generic failure for everything.
- The frontend shows these as readable messages in forms. For example, a failed checkout shows the real reason, like a stock issue, instead of a blank failure.